Final Project: Design a Long-Running Agent Workflow for a Finance Task

Individual project. Choose a substantial research or analytical activity and design an end-to-end agent workflow for it. The scope should justify multiple linked stages, repeated evaluation, persistent state, and recovery—not merely a single prompt or report update.

1. Specify a substantial, recurring task

Choose an activity from your work or study: **buy-side quantitative strategy research**, spanning hypotheses, data preparation, backtesting, robustness checks, portfolio construction, and ongoing monitoring; or **sell-side company/industry coverage**, spanning source collection, financial modeling, valuation, report production, and continuing updates. Other tasks, even those weakly related to finance, are welcome if comparable in scope. A single earnings update, valuation refresh, or calculation is only a subtask. Define the intended user, current process, stages and dependencies, recurring triggers, inputs, permitted sources, information cutoffs, work products, and acceptance criteria. Explain why this needs a persistent workflow rather than a one-shot script.

2. Design the architecture using the lecture concepts

Provide an annotated workflow diagram and a short design note explaining:

- **Roles and handoffs:** agent responsibilities, tools, reusable skills or instructions, and input/output artifacts. Justify the division of work; use deterministic code for calculations where appropriate. Distinguish agent decisions from the harness/controller that manages execution.
- **Long-running operation:** persistent state, checkpoints, scheduling or new-input triggers, dependencies, retries, and stop/escalation rules. Explain how an interrupted run resumes without repeating completed actions, and set limits on time, cost, and tool permissions.
- **Verification and improvement:** checks independent of the producing agent, human approval before consequential actions, and a bounded revision loop. Explain how a proposed change is tested against a fixed baseline, accepted or rejected, versioned, and rolled back. For time-sensitive tasks, prevent future-information leakage.

3. Make the design inspectable

Include one end-to-end example from input to work product, with intermediate artifacts and acceptance checks, plus one failure/interruption-and-recovery example. A small runnable prototype is welcome; a clearly labelled, step-by-step design walkthrough is acceptable. Distinguish executed results from simulated examples and identify limitations. Use public, synthetic, or properly licensed inputs; keep live trading, client publication, and other consequential actions simulated or behind explicit human approval.

4. Prepare the repository

Include your design note and diagram, prompts/skills/configuration, code if implemented, shareable example inputs and outputs, and verification/recovery evidence. The root `README.md` must give your identification, task summary, **repository URL**, file guide, and setup/run or walkthrough instructions (including dependencies where relevant). Disclose AI assistance and explain what you checked yourself. Use meaningful commits throughout development; do not submit only a final-file dump.

5. Submit one ZIP with complete Git history

GitHub, GitLab, or an equivalent Git host is acceptable. Commit and push the submission files and relevant development branches/tags, then archive a **full, non-shallow local clone, including its hidden `.git/` directory**. For example, on macOS/Linux, run from the parent folder:

```
git clone <repository-url> final-project
zip -r studentID-final-project.zip final-project
```

A host's "Download ZIP" or `git archive` is not sufficient: neither includes Git history. Extract your ZIP into a new folder and check that the README, all deliverables, and `.git/` are present. From the extracted repository root, `git log --all --oneline` must show the development history and `git fsck --full` must pass. The archive must be self-contained for reviewing files and history, without relying on repository access. Never include credentials, private client data, or secrets in files *or commit history*.